DBMS – ADVANCED SQL QUERY SYSTEM

Project Title: Gaming-Accessories E-Commerce Order Management System

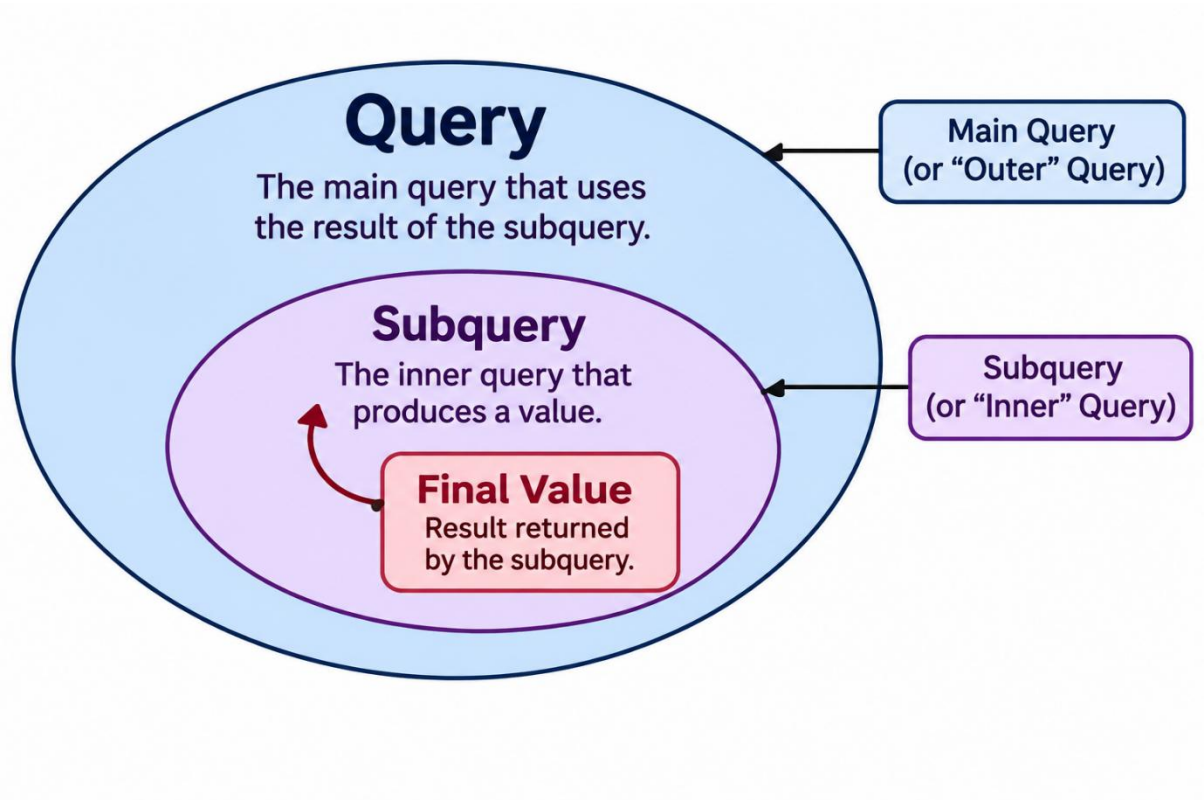
Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

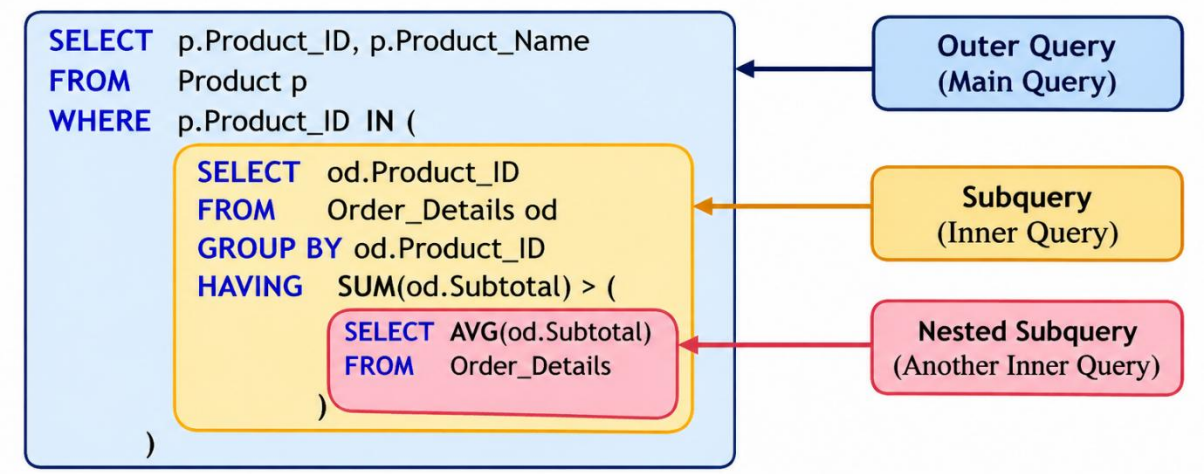
1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.



Basic Structure



Types of Subqueries

Type	Description
Single-row Subquery	Returns one value
Multiple-row Subquery	Returns multiple values
Nested Subquery	A subquery inside another subquery
Correlated Subquery	Inner query depends on the outer query

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
  2  FROM "ORDER"
  3  WHERE Total_Amount > (
  4      SELECT AVG(Total_Amount)
  5      FROM "ORDER"
  6  );

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
       101         101         2500
       103         103         3200
       105         105         2750
```

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price. Assuming the Product table contains:

- Product_ID
- Product_Name
- Price

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > (
  4      SELECT AVG(Price)
  5      FROM Product
  6  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       101
HP Laptop
     55000

       102
Dell Laptop
     62000
```

2.1 Products Below Average Price

The same concept can be used to find products below the average price.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < (
  4      SELECT AVG(Price)
  5      FROM Product
  6  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       103
Samsung Galaxy
     28000
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT Customer_ID,
  2         SUM(Total_Amount) AS Total_Purchase
  3  FROM "ORDER"
  4  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101           2500
        102           1800
        103           3200
        104           1500
        105           2750
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(Total_Purchase)
  2  FROM (
  3      SELECT SUM(Total_Amount) AS Total_Purchase
  4      FROM "ORDER"
  5      GROUP BY Customer_ID
  6  );

MAX(TOTAL_PURCHASE)
-------------------
               3200
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer01 c
  6  JOIN "ORDER" o
  7      ON c.Customer_ID = o.Customer_ID
  8  GROUP BY
  9      c.Customer_ID,
 10      c.First_Name,
 11      c.Last_Name
 12  HAVING SUM(o.Total_Amount) = (
 13      SELECT MAX(Total_Purchase)
 14      FROM (
 15          SELECT SUM(Total_Amount) AS Total_Purchase
 16          FROM "ORDER"
 17          GROUP BY Customer_ID
 18      )
 19  );

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
       103
Arun Raj
          3200
```

4. Complex Business Querieslated problems.

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Total_Amount
  6  FROM Customer01 c
  7  JOIN "ORDER" o
  8      ON c.Customer_ID = o.Customer_ID
  9  WHERE o.Total_Amount > (
 10      SELECT AVG(Total_Amount)
 11      FROM "ORDER"
 12  )
 13  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
        103
Arun Raj
       103         3200

        105
Karthik Mohan
       105         2750
```

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(od.Subtotal) AS Total_Sales
  5  FROM Product p
  6  JOIN Order_Details od
  7      ON p.Product_ID = od.Product_ID
  8  GROUP BY
  9      p.Product_ID,
 10      p.Product_Name
 11  HAVING SUM(od.Subtotal) > (
 12      SELECT AVG(Product_Sales)
 13      FROM (
 14          SELECT SUM(Subtotal) AS Product_Sales
 15          FROM Order_Details
 16          GROUP BY Product_ID
 17      )
 18  )
 19  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
       101
HP Laptop
       4700

       103
Samsung Galaxy
       4050
```

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      COUNT(o.Order_ID) AS Total_Orders,
  5      SUM(o.Total_Amount) AS Total_Purchase,
  6      AVG(o.Total_Amount) AS Average_Order_Value
  7  FROM Customer01 c
  8  JOIN "ORDER" o
  9      ON c.Customer_ID = o.Customer_ID
 10  GROUP BY
 11      c.Customer_ID,
 12      c.First_Name,
 13      c.Last_Name
 14  ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
------------ -------------- -------------------
         103
Arun Raj
           1           3200                3200

         105
Karthik Mohan
           1           2750                2750
```

Report Contains:

- Customer ID
- Customer Name
- Number of Orders
- Total Purchase Amount
- Average Order Value

7. Advanced Product Sales Report

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      COUNT(DISTINCT od.Order_ID) AS Number_Of_Orders,
  5      SUM(od.Quantity) AS Total_Quantity_Sold,
  6      SUM(od.Subtotal) AS Total_Sales,
  7      AVG(od.Unit_Price) AS Average_Price
  8  FROM Product p
  9  JOIN Order_Details od
 10      ON p.Product_ID = od.Product_ID
 11  GROUP BY
 12      p.Product_ID,
 13      p.Product_Name
 14  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------
         101
HP Laptop
                3                   7        4700           600

         103
Samsung Galaxy
                2                   4        4050        1012.5
```

This report combines product and order details.

Purpose: Displays:

- Number of orders for each product
- Total quantity sold
- Total sales
- Average selling price
- Products generating higher sales

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SQL> SELECT
  2      cat.Category_ID,
  3      cat.Category_Name,
  4      COUNT(DISTINCT od.Order_ID) AS Total_Orders,
  5      SUM(od.Quantity) AS Total_Quantity_Sold,
  6      SUM(od.Subtotal) AS Total_Sales,
  7      AVG(od.Unit_Price) AS Average_Price
  8  FROM Category cat
  9  JOIN Product p
 10      ON cat.Category_ID = p.Category_ID
 11  JOIN Order_Details od
 12      ON p.Product_ID = od.Product_ID
 13  GROUP BY
 14      cat.Category_ID,
 15      cat.Category_Name
 16  ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
------------ ------------------- ----------- -------------
           1
Laptops
            4                  11        7700           760

           2
Mobiles
            2                   4        4050        1012.5
```